MÓDULO BASES DE DATOS · SESIÓN 4 DE 5

MongoDB: los datos como objetos JS



Otra forma de guardar datos que se parece muchísimo a lo que ya sabes de JavaScript.

Bootcamp Full Stack · Clase online

DÓNDE ESTAMOS

Lo que ya sabes… y lo que viene



MySQL

Tablas, filas, columnas, JOIN.
¡Hecho!



Hoy

Otra forma de guardar los mismos datos



MongoDB

Documentos JSON, como los objetos de JS

01

NoSQL, Atlas y Compass

¿Y si los datos no encajan bien en tablas?

EL PROBLEMA

No todos los datos tienen la misma forma

👕 Una camiseta

- `nombre`: «Camiseta básica»
- `precio`: 12.99
- `tallas`: S, M, L, XL
- `colores`: blanco, negro
- `material`: algodón

💻 Un portátil

- `nombre`: «Portátil Pro 14»
- `precio`: 1199
- `ram`: 16 GB
- `procesador`: M3
- `puertos`: USB-C, HDMI

Un documento de MongoDB = un objeto JS

Mongo

```
{
  _id: ObjectId("66f1a2b3c4d5e6f7a8b9c0d1"),
  nombre: "Pikachu",
  tipo: "Eléctrico",
  stats: { ataque: 55, defensa: 40, velocidad: 90 },
  movimientos: ["Impactrueno", "Rayo"]
}
```

- Es **JSON**: ya lo conocéis de JavaScript
- Puede llevar **objetos dentro** (stats)
- Puede llevar **arrays** (movimientos)
- `_id` lo pone Mongo solo: es la **clave primaria**

CONCEPTOS

Diccionario MySQL → MongoDB

	MySQL		MongoDB
🗄️	Base de datos	→	Base de datos
📚	Tabla	→	Colección
📄	Fila	→	Documento
🏷️	Columna	→	Campo
🔑	PRIMARY KEY (id)	→	_id
🪆	JOIN	→	Objetos dentro de documentos

COMPARATIVA

El mismo Pokémon, dos formas de guardarlo

🐬 MySQL: 2 tablas + JOIN

🗂️ **pokemon**

id 🔑	nombre	tipo_id 🔗
25	Pikachu	4

🗂️ **tipos**

id 🔑	nombre
4	Eléctrico

🍃 MongoDB: 1 documento

```Mongo
{
  nombre: "Pikachu",
  tipo: "Eléctrico",
  stats: { ataque: 55,
           velocidad: 90 }
}
```

¿Cuándo usar cada una?

🐬 Relacional (MySQL)

- Datos con **estructura fija**
- Muchas **relaciones** entre tablas
- Todo tiene que cuadrar al céntimo
- Ej.: banco, facturas, reservas, stock

🍃 Documental (MongoDB)

- Datos **flexibles** o que cambian mucho
- Se leen **de una pieza** (un post con sus comentarios)
- Encaja de maravilla con **JavaScript**
- Ej.: catálogos, redes sociales, apps en tiempo real

COMPARATIVA

Cambia la forma de estructurar los datos

MySQL



MongoDB



HERRAMIENTAS

Atlas + Compass



Atlas

Tu MongoDB en la nube (gratis con el clúster M0). Ya creaste la cuenta en la tarea.



Compass

La app para ver y manejar tus datos. Tiene una consola (mongosh) abajo.



Cadena de conexión

mongodb+srv://usuario:contraseña@... Es la «dirección» de tu base de datos. ¡No la compartas!

👩‍💻 EJEMPLO GUIADO · PASO 1/5

Conectamos Compass a nuestro clúster de Atlas

1. En Atlas: botón Connect → opción Compass
2. Copia la cadena de conexión (mongodb+srv://...)
3. Abre Compass → New connection
4. Pega la cadena y cambia <password> por tu contraseña
5. Connect ✅

👩‍💻 EJEMPLO GUIADO · PASO 2/5

Creamos la base de datos y la colección

1. En Compass, botón «+» junto a Databases (Create database)
2. Database name: pokedex
3. Collection name: pokemon
4. Create Database ✅

👩‍💻 EJEMPLO GUIADO · PASO 3/5

Importamos los 49 Pokémon

1. Entra en la colección pokemon
2. Add data → Import JSON or CSV file
3. Elige pokemon.json
4. Import ✅ → ¡49 documentos!

👩‍🏫 EJEMPLO GUIADO · PASO 4/5

Filtramos desde la barra de Compass

Mongo

```
{ tipo: "Fuego" }
```

Resultado

nombre	tipo
Charizard	Fuego
Charmander	Fuego
Charmeleon	Fuego
Vulpix	Fuego
Ninetales	Fuego
Growlithe	Fuego
Arcanine	Fuego

↳ *10 filas (se muestran 7)*

👩‍🏫 EJEMPLO GUIADO · PASO 5/5

Insertamos uno a mano

Mongo

```
{
  "nombre": "Pichu",
  "tipo": "Eléctrico",
  "generacion": 2
}
```

💡 ¡No hace falta poner _id ni todos los campos! Cada documento puede ser diferente.

🛠️ EJERCICIO ⏱️ 20 min

Tu turno: tu colección de series 📺

1 Crea la base de datos streaming con la colección series

2 Importa series.json

3 Filtra las series de Netflix

4 Filtra las series españolas

5 Inserta 2 series a mano; una de ellas con un campo que no tenga ninguna otra (p. ej. premios: ["Emmy"])



Fíjate

Cuando insertes la serie con premios, mira las demás: ¡no tienen ese campo y no pasa nada! En MySQL habría que cambiar la tabla.

✅ CORRECCIÓN

Corrección del ejercicio 1 (1/2)

Mongo

```
// 1. Crea la base de datos streaming con la colección series
Create database → streaming / series

// 2. Importa series.json
Add data → Import JSON or CSV file → series.json

// 3. Filtra las series de Netflix
{ plataforma: "Netflix" }
```

✅ CORRECCIÓN

Corrección del ejercicio 1 (2/2)

Mongo

```
// 4. Filtra las series españolas
{ pais: "España" }

// 5. Inserta 2 series a mano; una de ellas con un campo que
// no tenga ninguna otra (p. ej. premios: ["Emmy"])
{
  titulo: "Fleabag", plataforma: "Prime Video",
  genero: "Comedia", anio_estreno: 2016, nota: 8.7,
  premios: ["Emmy", "Globo de Oro"]
}
```

02

Buscar documentos

find: el SELECT ... WHERE de MongoDB.

HERRAMIENTA

¿Dónde escribo los comandos?



Consola _MONGOSH

Abre la consola de abajo en Compass. Escribe use pokedex y ya estás dentro.



Barra Filter

Para filtros rápidos: pega solo el { ... } del find.



Truco

Flecha arriba ↑ recupera el comando anterior.

READ

find: buscar documentos

```mongo
// Todos
db.pokemon.find()

// Los que cumplen una condición
db.pokemon.find({ tipo: "Fuego" })

// Dos condiciones = AND
db.pokemon.find({ tipo: "Fuego", generacion: 1 })

// Solo el primero que encuentre
db.pokemon.findOne({ nombre: "Pikachu" })
```

- `db.coleccion.metodo(...)` siempre así
- El filtro es **un objeto**
- Varios campos = **AND**
- Textos entre **comillas** (dobles o simples)

READ

Operadores de comparación

$gt `>` mayor que	**$gte** `>=` mayor o igual	**$lt** `<` menor que
$lte `<=` menor o igual	**$ne** `<>` distinto de	**$in** `IN` está en la lista

```
db.pokemon.find({ peso_kg: { $gt: 100 } })   // peso > 100
```

READ

$or y campos dentro de objetos

Mongo

```
// Una cosa O la otra
db.pokemon.find({
  $or: [ { tipo: "Hielo" }, { tipo: "Dragón" } ]
})

// Campo dentro de un objeto: con punto y comillas
db.pokemon.find({ "stats.ataque": { $gte: 100 } })
```

- `$or` recibe un **array** de condiciones
- Para entrar en un objeto: `"stats.ataque"`
- Las comillas son **obligatorias** con el punto

CREATE

insertOne e insertMany

Mongo

```
db.pokemon.insertOne({
  nombre: "Pichu", tipo: "Eléctrico", generacion: 2
})

db.pokemon.insertMany([
  { nombre: "Cleffa", tipo: "Hada" },
  { nombre: "Igglybuff", tipo: "Normal" }
])
```

- **No hace falta** crear la colección antes
- **No hace falta** el `_id`
- `insertMany` recibe un **array**
- Cada documento puede tener **campos distintos**

💬 RESPONDE EN EL CHAT: A, B, C o D

¿Qué devuelve `db.series.find({ nota: { $lt: 7 } })`?

A Las series con nota 7

B Las series con nota menor que 7

C Las 7 primeras series

D Las series con nota mayor que 7

👩‍🏫 EJEMPLO GUIADO · PASO 1/7

Pokémon de tipo Fuego

Mongo

```
db.pokemon.find({ tipo: "Fuego" })
```

Resultado

nombre	tipo
Charizard	Fuego
Charmander	Fuego
Charmeleon	Fuego
Vulpix	Fuego
Ninetales	Fuego
Growlithe	Fuego
Arcanine	Fuego
Flareon	Fuego

↳ *10 filas (se muestran 8)*

👩‍🏫 EJEMPLO GUIADO · PASO 2/7

Pokémon de la 2ª generación

Mongo

```
db.pokemon.find({ generacion: 2 })
```

Resultado

nombre	generacion
Totodile	2
Cyndaquil	2
Togepi	2
Tyranitar	2
Umbreon	2
Lugia	2
Chikorita	2

↳ *7 filas*

👩‍🏫 EJEMPLO GUIADO · PASO 3/7

Ataque mayor que 110 (campo dentro de stats)

Mongo

```
db.pokemon.find({ "stats.ataque": { $gt: 110 } })
```

Resultado

nombre	stats.ataque
Gyarados	125
Flareon	130
Dragonite	134
Tyranitar	134
Garchomp	130

↳ *5 filas*

👩‍🏫 EJEMPLO GUIADO · PASO 4/7

Tipo Hielo o Dragón

Mongo

```
db.pokemon.find({ tipo: { $in: ["Hielo", "Dragón"] } })
```

Resultado

nombre	tipo
Articuno	Hielo
Dragonite	Dragón
Garchomp	Dragón
Dratini	Dragón

↳ *4 filas*

👩‍🏫 EJEMPLO GUIADO · PASO 5/7

Legendarios de tipo Psíquico (dos condiciones = AND)

Mongo

```
db.pokemon.find({ legendario: true, tipo: "Psíquico" })
```

Resultado

nombre	tipo	legendario
Mew	Psíquico	true
Lugia	Psíquico	true
Mewtwo	Psíquico	true

↳ *3 filas*

👩‍🏫 EJEMPLO GUIADO · PASO 6/7

Muy rápidos O muy pesados

Mongo

```
db.pokemon.find({
  $or: [
    { "stats.velocidad": { $gte: 120 } },
    { peso_kg: { $gt: 200 } }
  ]
})
```

Resultado

nombre	stats.velocidad	peso_kg
Alakazam	120	48
Onix	70	210
Gyarados	81	235
Lapras	60	220
Jolteon	130	24.5
Snorlax	30	460
Dragonite	80	210
Tyranitar	61	202

↳ *10 filas (se muestran 8)*

👩‍🏫 EJEMPLO GUIADO · PASO 7/7

Insertamos uno nuevo

Mongo

```
db.pokemon.insertOne({
  pokedex: 172, nombre: "Pichu",
  tipo: "Eléctrico", generacion: 2,
  stats: { ataque: 40, defensa: 15, velocidad: 60 }
})
```

💡 Compass responde con acknowledged: true y el _id que le ha puesto. Compruébalo con db.pokemon.find({ nombre: "Pichu" })

🛠️ EJERCICIO ⏱️ 20 min

Tu turno: busca en tus series 🔍

1. Series de la plataforma Max
2. Series con nota de 8.8 o más
3. Series estrenadas antes de 2015
4. Series de Comedia o de Animación
5. Series españolas ya finalizadas
6. Series con nota mayor que 9 O estrenadas en 2024
7. Inserta una serie nueva con insertOne



Pista

Usa la chuleta de operadores. Si algo falla, prueba primero el filtro en la barra Filter de Compass.

✅ CORRECCIÓN

Corrección del ejercicio 2 (1/3)

Mongo

```
// 1. Series de la plataforma Max
db.series.find({ plataforma: "Max" })

// 2. Series con nota de 8.8 o más
db.series.find({ nota: { $gte: 8.8 } })

// 3. Series estrenadas antes de 2015
db.series.find({ anio_estreno: { $lt: 2015 } })
```

✅ CORRECCIÓN

Corrección del ejercicio 2 (2/3)

Mongo

```
// 4. Series de Comedia o de Animación
db.series.find({ genero: { $in: ["Comedia", "Animación"] } })

// 5. Series españolas ya finalizadas
db.series.find({ pais: "España", finalizada: true })

// 6. Series con nota mayor que 9 O estrenadas en 2024
db.series.find({
  $or: [{ nota: { $gt: 9 } }, { anio_estreno: 2024 }]
})
```

✅ CORRECCIÓN

Corrección del ejercicio 2 (3/3)

Mongo

```
// 7. Inserta una serie nueva con insertOne
db.series.insertOne({ titulo: "Shogun", plataforma: "Disney+",
  genero: "Drama", anio_estreno: 2024, nota: 8.6 })
```

03

Ordenar, limitar y contar

El Top 5 en versión MongoDB.

READ

Elegir campos, ordenar y limitar

Mongo

```
// Solo nombre y tipo (1 = sí, 0 = no)
db.pokemon.find({}, { nombre: 1, tipo: 1, _id: 0 })

// Ordenar: 1 = ascendente, -1 = descendente
db.pokemon.find().sort({ peso_kg: -1 })

// Los 3 primeros
db.pokemon.find().sort({ peso_kg: -1 }).limit(3)
```

- 2º parámetro de find = **qué campos**
- `.sort()` = ORDER BY
- `.limit()` = LIMIT
- Se **encadenan** como en JS

READ

Contar y valores distintos

Mongo

```
db.pokemon.countDocuments()                  // todos
db.pokemon.countDocuments({ tipo: "Agua" }) // con filtro

db.pokemon.distinct("tipo")                  // sin repetidos
```

- `countDocuments` = `SELECT COUNT(*)`
- `distinct` = `SELECT DISTINCT`

RESUMEN

Chuleta SQL ↔ MongoDB

Quiero…	🐬 SQL	🍃 MongoDB
Todo	`SELECT * FROM pokemon`	`db.pokemon.find()`
Filtrar	`WHERE tipo = 'Agua'`	`find({ tipo: "Agua" })`
Comparar	`WHERE ataque >= 100`	`find({ ataque: { $gte: 100 } })`
Columnas	`SELECT nombre, tipo`	`find({}, { nombre: 1, tipo: 1 })`
Ordenar	`ORDER BY ataque DESC`	`.sort({ ataque: -1 })`
Limitar	`LIMIT 5`	`.limit(5)`
Contar	`SELECT COUNT(*)`	`.countDocuments()`

👩‍🏫 EJEMPLO GUIADO · PASO 1/5

Solo nombre y tipo (proyección)

Mongo

```
db.pokemon.find({}, { nombre: 1, tipo: 1, _id: 0 })
```

Resultado

nombre	tipo
Wartortle	Agua
Bulbasaur	Planta
Charizard	Fuego
Charmander	Fuego
Charmeleon	Fuego
Vulpix	Fuego
Blastoise	Agua
Pikachu	Eléctrico

↳ *49 filas (se muestran 8)*

👩‍🏫 EJEMPLO GUIADO · PASO 2/5

Los 5 con más ataque

Mongo

```
db.pokemon.find({}, { nombre: 1, "stats.ataque": 1, _id: 0 })
  .sort({ "stats.ataque": -1 })
  .limit(5)
```

Resultado

nombre	stats.ataque
Tyranitar	134
Dragonite	134
Garchomp	130
Flareon	130
Gyarados	125

↳ *5 filas*

👩‍🏫 EJEMPLO GUIADO · PASO 3/5

¿Cuántos son de tipo Agua?

Mongo

```
db.pokemon.countDocuments({ tipo: "Agua" })
```

Resultado

resultado
9

↳ *1 fila*

👩‍🏫 EJEMPLO GUIADO · PASO 4/5

¿Qué tipos distintos hay?

Mongo

```
db.pokemon.distinct("tipo")
```

Resultado

valor
Agua
Dragón
Eléctrico
Fantasma
Fuego
Hada
Hielo
Lucha

↳ *14 filas (se muestran 8)*

👩‍🏫 EJEMPLO GUIADO · PASO 5/5

Los 3 legendarios más rápidos

Mongo

```
db.pokemon.find({ legendario: true },
                { nombre: 1, "stats.velocidad": 1, _id: 0 })
  .sort({ "stats.velocidad": -1 })
  .limit(3)
```

Resultado

nombre	stats.velocidad
Mewtwo	130
Lugia	110
Mew	100

↳ *3 filas*

🛠️ EJERCICIO ⏱️ 25 min 🤖 Con IA

Mongo Quest 🎮

1 🥉 Título y nota de todas las series (sin _id)

2 🥉 ¿Cuántas series hay de Netflix?

3 🥈 Las 5 mejores series (título y nota)

4 🥈 ¿Qué plataformas distintas hay?

5 🥇 Las 3 series españolas más recientes

6 💎 ¿Cuántas series de Drama tienen nota de 8.5 o más?



Cómo se juega

🥉 = 1 punto, 🥈 = 2, 🥇 = 3, 💎 = 5.
Podéis pedir a la IA una pista (no la solución) a cambio de 1 punto.
¡Gana la pareja con más puntos!

✅ CORRECCIÓN

Corrección de Mongo Quest (1/2)

Mongo

```
// 1. 🥉 Título y nota de todas las series (sin _id)
db.series.find({}, { titulo: 1, nota: 1, _id: 0 })

// 2. 🥉 ¿Cuántas series hay de Netflix?
db.series.countDocuments({ plataforma: "Netflix" })

// 3. 🥈 Las 5 mejores series (título y nota)
db.series.find({}, { titulo: 1, nota: 1, _id: 0 })
  .sort({ nota: -1 }).limit(5)
```

✅ CORRECCIÓN

Corrección de Mongo Quest (2/2)

Mongo

```
// 4. 🥈 ¿Qué plataformas distintas hay?
db.series.distinct("plataforma")

// 5. 🥇 Las 3 series españolas más recientes
db.series.find({ pais: "España" }, { titulo: 1, anio_estreno: 1, _id: 0 })
  .sort({ anio_estreno: -1 }).limit(3)

// 6. 💎 ¿Cuántas series de Drama tienen nota de 8.5 o más?
db.series.countDocuments({ genero: "Drama", nota: { $gte: 8.5 } })
```

💬 RESPONDE EN EL CHAT: A, B, C o D

¿Cuál es el equivalente en MongoDB de SELECT titulo FROM series ORDER BY nota DESC LIMIT 1 ?

A

```
db.series.find({ titulo: 1
}).limit(1)
```

B

```
db.series.find({}, { titulo: 1
}).sort({ nota: -1 }).limit(1)
```

C

```
db.series.findOne({ nota: -1 })
```

D

```
db.series.sort({ nota: 1
}).titulo
```

MÓDULO BASES DE DATOS · SESIÓN 5 DE 5

CRUD completo y una app real



Modificar y borrar documentos... y ver MongoDB funcionando en una web en la que escribe toda la clase.

Bootcamp Full Stack · Clase online

REPASO

CRUD: las 4 operaciones de cualquier app

➕

Create

SQL: INSERT
Mongo: insertOne

👀

Read

SQL: SELECT
Mongo: find

✏️

Update

SQL: UPDATE
Mongo: updateOne

🗑️

Delete

SQL: DELETE
Mongo: deleteOne

01

Modificar y borrar

updateOne, updateMany, deleteOne y deleteMany.

UPDATE

updateOne y updateMany

Mongo

```
db.pokemon.updateOne(
  { nombre: "Pikachu" },          // ¿cuál?   (filtro)
  { $set: { "stats.ataque": 60 } } // ¿qué cambio?
)

db.pokemon.updateMany(
  { generacion: 1 },
  { $set: { region: "Kanto" } }
)
```

- 1º parámetro: el **filtro** (como el WHERE)
- 2º parámetro: **qué cambiar**
- `updateOne` → el **primero** que encuentre
- `updateMany` → **todos** los que cumplan

UPDATE

Operadores de actualización



$set

Pone un valor (o crea el campo).
{ $set: { nota: 9 } }



$inc

Suma o resta.
{ $inc: { likes: 1 } }



$push

Añade a un array.
{ $push: { tags: "top" } }



$unset

Elimina un campo.
{ $unset: { tags: "" } }

DELETE

deleteOne y deleteMany

Mongo

```
db.pokemon.deleteOne({ nombre: "Magikarp" })

db.pokemon.deleteMany({ "stats.ataque": { $lt: 30 } })

// ⚠️ Filtro vacío = borra TODOS
db.pokemon.deleteMany({})
```

- Mismo filtro que en `find`
- Regla de oro (¡como en SQL!): **find primero** con el mismo filtro
- `{}` vacío = **todos** ⚠

💬 RESPONDE EN EL CHAT: A, B, C o D

¿Qué hace db.series.updateMany({}, { $inc: { vistas: 1 } }) ?

A Suma 1 a «vistas» solo en la primera serie

B Suma 1 a «vistas» en TODAS las series

C Da error porque el filtro está vacío

D Pone «vistas» a 1 en todas

👩‍🏫 EJEMPLO GUIADO · PASO 1/6

Paso 0 de la regla de oro: miramos ANTES

Mongo

```
db.pokemon.find({ nombre: "Pikachu" })
```

Resultado

nombre	stats.ataque	movimientos
Pikachu	55	

↳ *1 fila*

👩‍🏫 EJEMPLO GUIADO · PASO 2/6

Subimos el ataque de Pikachu ($set)

Mongo

```
db.pokemon.updateOne(
  { nombre: "Pikachu" },
  { $set: { "stats.ataque": 60 } }
)
```

Resultado

nombre	stats.ataque	movimientos
Pikachu	60	

↳ *1 fila*

👩‍🏫 EJEMPLO GUIADO · PASO 3/6

Le enseñamos un movimiento ($push)

Mongo

```
db.pokemon.updateOne(
  { nombre: "Pikachu" },
  { $push: { movimientos: "Impactrueno" } }
)
```

💡 El campo movimientos no existía: $push lo crea como array.

Resultado

nombre	stats.ataque	movimientos
Pikachu	60	Impactrueno

↳ *1 fila*

👩‍🏫 EJEMPLO GUIADO · PASO 4/6

+10 de ataque a TODOS los de Fuego ($inc)

Mongo

```
db.pokemon.updateMany(
  { tipo: "Fuego" },
  { $inc: { "stats.ataque": 10 } }
)
```

Resultado

nombre	stats.ataque
Charmander	62
Charizard	94
Vulpix	51
Ninetales	86
Arcanine	120
Growlithe	80
Charmeleon	74
Moltres	110

↳ *10 filas (se muestran 8)*

👩‍🏫 EJEMPLO GUIADO · PASO 5/6

Añadimos la región a los de 1ª generación

Mongo

```
db.pokemon.updateMany(
  { generacion: 1 },
  { $set: { region: "Kanto" } }
)
```

Resultado

total
40

↳ *1 fila*

👩‍🏫 EJEMPLO GUIADO · PASO 6/6

Adiós, Magikarp (deleteOne)

Mongo

```
db.pokemon.deleteOne({ nombre: "Magikarp" })
```

💡 49 − 1 = 48. Si hubiera borrado con un filtro vacío, quedarían 0…

Resultado

total
48

↳ *1 fila*

🛠️ EJERCICIO ⏱️ 20 min

Tu turno: actualiza tus series ✏️

1. Cambia la nota de «Dark» a 9.0
2. Marca «Arcane» como favorita (campo nuevo favorita: true)
3. Suma una temporada a «Miércoles» ($inc)
4. Añade la etiqueta «superhéroes» a «The Boys» ($push)
5. Marca como finalizadas todas las series anteriores a 2015
6. Borra las series con nota menor que 7.3 (¡find primero!)



Recuerda

Antes de cada update o delete, ejecuta un find con el mismo filtro. ¿Salen solo los documentos que quieres?

✅ CORRECCIÓN

Corrección del ejercicio 1 (1/2)

Mongo

```
// 1. Cambia la nota de «Dark» a 9.0
db.series.updateOne({ titulo: "Dark" }, { $set: { nota: 9.0 } })

// 2. Marca «Arcane» como favorita (campo nuevo favorita:
// true)
db.series.updateOne({ titulo: "Arcane" }, { $set: { favorita: true } })

// 3. Suma una temporada a «Miércoles» ($inc)
db.series.updateOne({ titulo: "Miércoles" }, { $inc: { temporadas: 1 } })
```

✅ CORRECCIÓN

Corrección del ejercicio 1 (2/2)

Mongo

```
// 4. Añade la etiqueta «superhéroes» a «The Boys» ($push)
db.series.updateOne({ titulo: "The Boys" },
  { $push: { etiquetas: "superhéroes" } })

// 5. Marca como finalizadas todas las series anteriores a
// 2015
db.series.updateMany({ anio_estreno: { $lt: 2015 } },
  { $set: { finalizada: true } })

// 6. Borra las series con nota menor que 7.3 (¡find
// primero!)
db.series.find({ nota: { $lt: 7.3 } })
db.series.deleteMany({ nota: { $lt: 7.3 } })
```

02

¿Embeber o referenciar?

Cómo organizar los datos en MongoDB.

MODELADO

Dos formas de guardar las capturas de Ash

🪆 Embeber (dentro)

Mongo

```
{ nombre: "Ash",
  capturas: [
    { pokemon: "Pikachu", nivel: 35 },
    { pokemon: "Snorlax", nivel: 40 }
  ] }
```

- Se lee **todo de una vez**
- Ideal si los datos **van siempre juntos**

🔗 Referenciar (con _id)

Mongo

```
// colección capturas
{ entrenador_id: ObjectId("a1…"),
  pokemon: "Pikachu", nivel: 35 }

// colección entrenadores
{ _id: ObjectId("a1…"), nombre: "Ash" }
```

- Como una **FK** de MySQL
- Ideal si el dato se **comparte** o **crece mucho**

MODELADO

Regla práctica para decidir



Embebe si…

Se leen siempre juntos (un post y sus 10 comentarios) y no crecen sin control.



Referencia si…

El dato se comparte entre muchos (un vendedor con 500 productos) o cambia a menudo.



¿Dudas?

Pregúntate: ¿cómo lo voy a mostrar en la web? Guárdalo como lo vas a leer.

👩‍🏫 EJEMPLO GUIADO · PASO 1/3

Guardamos a Ash con sus capturas DENTRO

Mongo

```
db.entrenadores.insertOne({
  nombre: "Ash",
  ciudad: "Pueblo Paleta",
  capturas: [
    { pokemon: "Pikachu",   apodo: "Pika", nivel: 35 },
    { pokemon: "Charmander", nivel: 30 },
    { pokemon: "Snorlax",   nivel: 40 }
  ]
})
```

Resultado

total entrenadores
1

↳ *1 fila*

👩‍🏫 EJEMPLO GUIADO · PASO 2/3

¿Qué entrenadores tienen a Pikachu?

Mongo

```
db.entrenadores.find({ "capturas.pokemon": "Pikachu" })
```

💡 Para buscar dentro de un array de objetos se usa el punto: "capturas.pokemon".

Resultado

nombre	ciudad
Ash	Pueblo Paleta

↳ *1 fila*

👩‍🏫 EJEMPLO GUIADO · PASO 3/3

Atrapa uno nuevo: $push al array

Mongo

```
db.entrenadores.updateOne(
  { nombre: "Ash" },
  { $push: { capturas: { pokemon: "Lucario", nivel: 45 } } }
)
```

Resultado

nombre	capturas
Ash	Pikachu, Charmander, Snorlax, Lucario

↳ *1 fila*

🛠️ EJERCICIO ⏱️ 20 min 🤖 Con IA

Tu turno: EcoSwap en MongoDB ♻️

1 Recuerda el diseño de **EcoSwap** de la sesión 2 (usuarios, productos, categorías, favoritos)

2 Decide: ¿qué **colecciones** tendrías? ¿Qué **embebes** y qué **referencias**?

3 Pide a la IA su propuesta de documentos y **compárala** con la tuya

4 Escribe un documento de ejemplo de cada colección e **insértalo** en Atlas



Prompt

«Tengo este diseño relacional: [pega tu diagrama]. ¿Cómo lo modelarías en MongoDB? Dime qué embeber y qué referenciar y por qué.»

CORRECCIÓN

Una posible solución

Mongo

```
// colección usuarios
{ _id: ObjectId("u1"), nombre: "Lucía", email: "lucia@mail.com",
  ciudad: "Madrid", favoritos: [ObjectId("p7"), ObjectId("p9")] }

// colección productos
{ _id: ObjectId("p7"), nombre: "Bici de paseo", precio: 80,
  categoria: "Deportes", estado: "En venta",
  vendedor_id: ObjectId("u2"), fecha: ISODate("2025-10-01") }
```

- **Referencia** al vendedor: un usuario tiene muchos productos
- **Categoría** como texto: son pocas y casi no cambian
- **Favoritos** como array de ids dentro del usuario

03

El muro de la clase

Una app web de verdad, conectada a la misma base de datos para todos.

ARQUITECTURA

Así funciona el muro



Navegador

Escribes un mensaje, das ❤ o borras



server.js

Ya está hecho: recibe la petición y llama a db.js



db.js

¡Aquí programas TÚ el CRUD!



Atlas

Guarda los mensajes de toda la clase

🎉 Todos conectados a la MISMA base de datos: lo que escribas aparecerá en el muro de tus compañeros.

DEL TERMINAL AL CÓDIGO

mongosh vs Node.js: casi igual

En mongosh (Compass)

- `db.mensajes.find()`
- Los resultados salen solos
- `_id` se escribe tal cual

En Node.js (db.js)

- `await mensajes.find().toArray()`
- Hay que poner **await** (espera a la base de datos)
- El id llega como texto: `new ObjectId(id)`

PRÁCTICA FINAL

Pon en marcha el kit



1. Descarga

El enlace está en el chat. Ábrelo con VS Code.



2. Configura .env

Pega la cadena de conexión que te paso por el chat.



3. Arranca

npm install y después npm start



4. Abre

http://localhost:3000
¡Aún no funciona nada!

👩‍🏫 EJEMPLO GUIADO · PASO 1/2

El profe completa crearMensaje

JS

```
const { MongoClient, ObjectId } = require('mongodb');
const client = new MongoClient(process.env.MONGODB_URI);
const mensajes = client.db('muro').collection('mensajes');

async function crearMensaje(texto, autor) {
  await mensajes.insertOne({ texto, autor, likes: 0, fecha: new Date() });
}
```

💡 Escribe un mensaje en la web y búscalo en Compass: base de datos muro → colección mensajes.

👩‍🏫 EJEMPLO GUIADO · PASO 2/2

…y listarMensajes: ¡aparecen los post-its!

JS

```
async function listarMensajes() {
  return mensajes.find().sort({ fecha: -1 }).toArray();
}
```

💡 find() + sort + toArray(). Recarga la web: ¡se ven los mensajes de toda la clase! 🎉

🛠️ EJERCICIO ⏱️ 35 min 🤖 Con IA

Tu turno: termina el muro 📌

1. darLike(id): suma 1 a likes del mensaje con ese _id
2. borrarMensaje(id): borra el mensaje con ese _id
3. Extra: editarMensaje(id, texto) con $set
4. Extra: listarDeAutor(autor), solo los mensajes de una persona
5. Prueba cada función en la web y **mira el cambio en Compass**



Usa la IA como tutor

«Actúa como mi profesor. No me des la solución: dame pistas para completar esta función de MongoDB en Node: [pega la función]»

✅ CORRECCIÓN

Corrección: el muro completo (1/2)

JS

```
// 1. darLike(id): suma 1 a likes del mensaje con ese _id
async function darLike(id) {
  await mensajes.updateOne(
    { _id: new ObjectId(id) },
    { $inc: { likes: 1 } }
  );
}

// 2. borrarMensaje(id): borra el mensaje con ese _id
async function borrarMensaje(id) {
  await mensajes.deleteOne({ _id: new ObjectId(id) });
}
```

✅ CORRECCIÓN

Corrección: el muro completo (2/2)

JS

```
// 3. Extra: editarMensaje(id, texto) con $set
async function editarMensaje(id, texto) {
  await mensajes.updateOne(
    { _id: new ObjectId(id) },
    { $set: { texto } }
  );
}

// 4. Extra: listarDeAutor(autor), solo los mensajes de una
// persona
async function listarDeAutor(autor) {
  return mensajes.find({ autor }).sort({ fecha: -1 }).toArray();
}
```



Cierre del módulo

Lo que ya sabes hacer.

RESUMEN

Lo que ya sabes hacer 🏆

🔍

Consultar

SELECT, WHERE, ORDER BY, GROUP BY, JOIN

📐

Diseñar

Tablas, claves y relaciones (y revisar a la IA)

🏗️

Construir

CREATE, INSERT, UPDATE y DELETE con seguridad

🍃

MongoDB

Documentos, find, CRUD y embeber o referenciar

🚀

Una app real

Node.js conectado a una base de datos

🤖

Con IA

Pedir bien, leer y comprobar

PARA LLEVARTE

Cómo trabajar con la IA y las bases de datos



Dale la estructura

CREATE TABLE o un documento de ejemplo. Sin contexto, se inventa columnas.



Lee antes de ejecutar

¿Hay WHERE? ¿El filtro es el correcto? ¿Las FK existen?



Comprueba el resultado

¿Tiene sentido? Contrástalo con algo que sepas.



Nunca a ciegas

UPDATE, DELETE y DROP de la IA: SIEMPRE revisados primero.

PRÓXIMOS PASOS

¿Y ahora qué?



Node.js

Construiréis vosotros el backend que hoy os dábamos hecho



React

Un frontend que pide datos a vuestro backend



Proyecto EcoSwap

Opcional: el marketplace con MongoDB

🎉 Ya sabéis dónde viven los datos de una app y cómo hablar con ellos. ¡Enhorabuena!